Scrollspy / Table of Contents

- **Problem:** Build a documentation-style page with a long article (h2 sections and h3 sub-sections) and a sticky **table of contents** (TOC) next to it. While the user scrolls, the TOC **highlights the section being read**. Clicking a TOC link **smoothly scrolls** to that section and updates the URL hash. Opening the page with `#some-section` in the URL lands on that section.
- **Tech stack:** React (Vite, JavaScript, Tailwind CSS v4)
- **Time limit:** 60 minutes
- **Often asked at:** Google, Stripe
- **Why it matters:** IntersectionObserver

PART A: Thought process

General steps for any machine coding task

1. **Understand the requirements.** Restate the problem, split functional vs non-functional, ask clarifying questions, write down assumptions.
2. **List the interactions.** Every user action and its expected result. This is your checklist for building and testing.
3. **Design the architecture.** Name the components and their one-line jobs. Start small.
4. **Design the state.** What is the single source of truth? What can be derived?
5. **Design data structures and contracts.** Shape of the data, component props, hook API.
6. **Trace one data flow end to end.** Proves the design works before you type.
7. **Find the hard parts.** Solve them on paper first.
8. **Build in layers.** Simplest working version first, then one working layer at a time.
9. **Cover states and edge cases.** Top of page, bottom of page, deep link, fast clicks, short sections.
10. **Wrap up.** Trade-offs and what you would add with more time.

Time budget for 60 minutes

Phase	Time
Requirements + clarifying questions	5 min
Interactions, architecture, state design	7 min
Static layout: article + sticky TOC + plain anchor links	10 min
`useScrollSpy` with IntersectionObserver (the core)	15 min
Click-to-scroll lock, hash, deep link	10 min
Edge cases (last section, header offset, a11y)	8 min
Wrap-up and trade-offs	5 min

Think out loud

Say what you are deciding and why: "I will not listen to `scroll` and measure every heading. I'll let IntersectionObserver tell me when a section crosses a reading line just below the header, and keep the active id in one place." The interviewer grades the reasoning as much as the result. When you take a shortcut, name it.

PART B: Apply it to this problem

1. Understand the requirements

Functional requirements

- Render a long article made of sections. Each section has a heading (h2 or h3) and paragraphs.
- Render a TOC with one link per section. h3 links are indented under their h2.
- The TOC stays visible while scrolling (sticky).
- **Scrollspy:** the link of the section currently being read is highlighted. The h2 parent of an active h3 is slightly emphasized.
- **Click to navigate:** clicking a TOC link smoothly scrolls to the section, the heading is not hidden under the sticky header, the clicked link is highlighted at once and stays highlighted (no flicker through the sections in between).
- **URL hash:** after a click the URL becomes `/#section-id`, so the link can be shared.
- **Deep link:** loading `/#section-id` opens the page at that section with it highlighted.
- At the very bottom of the page the last section is highlighted, even if it is too short to reach the top.

Non-functional requirements

Area	What it means here
Performance	No work on every scroll pixel. No layout reads (`getBoundingClientRect`) in a scroll handler. Only re-render when the active id actually changes.
Scalability	Works the same with 10 or 200 sections: one observer for all sections.
Error handling	Missing element for an id, unknown hash, empty article: no crash, nothing highlighted wrongly.
Accessibility	TOC is a `<nav aria-label>` with a list of real `<a href="#id">` links, active link has `aria-current="location"`, focus moves to the target section after a click, respects `prefers-reduced-motion`.
Usability	Sticky header offset handled, cmd/ctrl-click still opens a new tab, the TOC keeps its active link in view when it is long.
Reusability	Scrollspy logic in a `useScrollSpy(ids, options)` hook that knows nothing about the TOC markup.
Extensibility	Content comes as data, so it can later come from markdown / a CMS. Header height and reading line are options.

Clarifying questions to ask the interviewer

1. **Where does the content come from?** A static array, markdown, or headings already in the DOM? (Decides whether we build the TOC from data or scan the DOM.)
2. **How many heading levels?** Only h2, or h2 + h3? Do we need collapsible groups?
3. **Which section counts as "active"** when two are on screen: the one at the top, or the one that takes most of the viewport?
4. **Is there a sticky header?** How tall? (Affects the scroll offset and the reading line.)
5. **Should the URL hash change while scrolling, or only on click?** And should each click add a browser history entry?
6. **Mobile?** Hide the TOC, or show a collapsible "On this page" dropdown?

Assumptions we will build with

- Content is a local array of `{ title, level, paragraphs }`. Levels 2 and 3 only. Ids are generated from titles.
- **Active = the first section (in document order) that crosses a reading band just below the header** (the top 40% of the viewport). This matches how docs sites (MDN, Stripe docs) behave.
- Sticky header of 64px.
- The hash changes **only on click** (with `replaceState`, so Back does not walk through every TOC click). Scrolling does not touch the URL.
- TOC is shown on large screens only (`lg:`). Mobile TOC is a "more time" item.
- Modern browsers: IntersectionObserver everywhere, `scrollend` where available with a timeout fallback.

---

2. Identify interactions / use cases

Action / input	Expected behavior / output
Page loads at the top	First section highlighted
Scroll down slowly	The highlight moves to each section when its top crosses the reading line below the header
Scroll back up	The highlight moves back the same way
Stop in the middle of a long section (no heading on screen)	That section stays highlighted
Reach the very bottom, last section is short	Last section highlighted
Active section is an h3	The h3 link is highlighted, its parent h2 link is emphasized
Click a TOC link	Clicked link highlighted at once, page smoothly scrolls, heading lands just below the header, URL becomes `#id`, focus moves to the section
Click a link far away (sections in between pass the reading line)	Highlight does NOT flicker through the in-between sections

Action / input	Expected behavior / output
Click link A, then quickly link B	Ends on B, highlight on B
User scrolls with the wheel during a smooth scroll	Scrolling continues; the highlight follows again once the scroll ends
Cmd/Ctrl-click a TOC link	Browser opens the link in a new tab (default behavior kept)
Load page with `#picking-the-active-section`	Page opens at that section, it is highlighted
Load page with an unknown hash `#nope`	Normal load at the top, first section highlighted
User has reduced motion enabled	Clicks jump instantly instead of smooth scrolling
Long TOC, active link off-screen inside the TOC	TOC scrolls itself so the active link is visible
Two sections with the same title	Each gets a unique id, both links work

3. Design the architecture

Piece	Responsibility
`App`	Builds sections with ids, owns the layout (header, article, aside), calls `useScrollSpy`, handles the deep link once
`Article`	Renders sections as `<section id>` with h2/h3 headings, plus an end sentinel
`TableOfContents`	Renders the links, highlights `activeId`, calls `onSelect(id)` on click, keeps the active link in view
`useScrollSpy(ids, { offset, endRef })`	Observes sections, decides `activeId`, exposes `scrollTo(id)` with the click lock
`utils/toc.js`	Pure functions: `slugify`, `withIds` (unique ids + parent ids), `pickActive`
`data/article.js`	Mock content

```
App
 ├── useScrollSpy(ids, { offset, endRef })  ->  { activeId, scrollTo }
 ├── <header>                        (sticky, 64px)
 ├── <main>
 │    └── Article(sections, endRef)
 │          ├── <section id=...> h2/h3 + paragraphs   (observed)
 │          └── <div ref=endRef />                    (end sentinel, observed)
 └── <aside>
      └── TableOfContents(items, activeId, onSelect=scrollTo)
```

Why this split: the **hook** is the only piece that touches observers and scrolling; the **TOC** is a dumb list that could be swapped for a progress bar or a mobile dropdown; the **pure utils** are where the bugs usually hide (ids, picking), so they are testable without a browser.

---

4. Design the state / data model

State or field	Purpose
`activeId` (React state, in the hook)	The id of the highlighted section. The only value that causes a re-render.
`visibleRef: Map<id, boolean>` (ref)	Which sections currently cross the reading band. Updated by the observer, never rendered.
`atEndRef: boolean` (ref)	Whether the end of the article is on screen (bottom-of-page rule).
`lockedRef: boolean` (ref)	True while a click-triggered smooth scroll is running: the observer must not change `activeId`.
`releaseRef: function` (ref)	Cleans up the current lock (listener + timer); lets a new click cancel the old lock.
`sections` (derived with `useMemo`)	Content with `id` and `parentId` added.
`ids` (derived with `useMemo`)	Section ids in document order: input for the hook.

```
const [activeId, setActiveId] = useState(ids[0] ?? null);
const visibleRef = useRef(new Map()); // id -> isIntersecting
const atEndRef = useRef(false);
const lockedRef = useRef(false);
const releaseRef = useRef(null);
```

Senior note: derive, don't store

- **Refs vs state:** the visibility map changes many times during a scroll, but the UI only cares about the result. Keeping it in a ref and calling `setActiveId` only when the picked id changes means the TOC re-renders a handful of times per page, not on every observer callback. React also bails out when `setActiveId` gets the same value.
- **Derived:** `ids`, `parentId`, `isActive`, `isParentOfActive`, the indentation: all computed from `sections` + `activeId`. Nothing about them is stored twice.
- **Ownership:** the hook owns `activeId` (it is the only writer, through the observer or `scrollTo`). `App` owns the content. The TOC owns nothing but its own scroll position.
- **Not in state:** the URL hash. The browser already stores it; we write it with `history.replaceState` and read it once on load.

---

5. Design the data structures and contracts

Example data

```
// input (from CMS / markdown)
{ title: 'Why IntersectionObserver?', level: 2, paragraphs: ['...'] }

// after withIds()
{ title: 'Why IntersectionObserver?', level: 2, paragraphs: ['...'],
  id: 'why-intersectionobserver', parentId: null }
{ title: 'Scroll events', level: 3, paragraphs: ['...'],
  id: 'scroll-events', parentId: 'why-intersectionobserver' }
```

- **Ids come from the title** (slug), so links are readable and stable across reloads: `#scroll-events` keeps working after the content is re-ordered. Array indexes would break shared links.
- **Duplicates get a suffix:** two "Edge cases" headings become `edge-cases` and `edge-cases-1`.
- **A flat list with `level` + `parentId`** instead of a nested tree: the article and the TOC both render in document order, and "first visible in document order" is just `ids.find(...)`. The tree relationship we need (parent emphasis) is one field.
- **Each `<section>` wraps the heading and its own paragraphs only** (h3 sections are siblings of h2 sections, not children). The sections tile the page with no overlap, which makes "which one is at the reading line" unambiguous.

Contracts

```
// pure utils
slugify(text: string) => string                          // 'Café & Crème' -> 'cafe-creme'
withIds(sections) => sections with { id, parentId }
pickActive(ids: string[], visible: Map<string, boolean>, atEnd: boolean) => string | null
                                                          // null = keep the previous active id

// hook
useScrollSpy(ids: string[], { offset?: number, endRef?: RefObject })
  => { activeId: string | null, scrollTo(id: string, { smooth?: boolean }): void }

// components
<Article title sections endRef />
<TableOfContents items activeId onSelect={(id) => void} />
```

No API here: the mock array stands in for the CMS response. If content came from an API, `App` would fetch it, show a skeleton while loading, and only render `Article` + call the hook once `sections` exist (the hook already handles `ids = []`).

---

6. Trace the data flow

Interaction 1: the user scrolls down into "Observer basics"

```
User scrolls (wheel / touch / keyboard)
  │
  ▼
Browser moves the page (no JS runs per pixel)
  │
  ▼
"Observer basics" <section> top crosses the reading line (64 + 8 = 72px from the top)
  │
  ▼
IntersectionObserver callback: entries = [{ target: observer-basics, isIntersecting: true },
                                          { target: scroll-events,  isIntersecting: false }]
  │   visible.set('observer-basics', true); visible.set('scroll-events', false)
  ▼
lockedRef false -> pickActive(ids, visible, atEnd=false)
  │   = first id in document order with visible === true = 'observer-basics'
  ▼
setActiveId('observer-basics')   (React skips the render if it was already active)
  │
  ▼
TableOfContents re-renders: link gets aria-current="location" + blue border,
parent 'why-intersectionobserver' gets darker text, TOC scrolls itself if needed
```

Interaction 2: the user clicks "Summary" while at the top

```
click on <a href="#summary">  ->  preventDefault()  ->  scrollTo('summary')
  │
  ▼
release previous lock (if any) -> lockedRef = true -> setActiveId('summary')   (instant feedback)
  │
  ▼
listen for 'scrollend' + 1s fallback timer  ->  el.scrollIntoView({ behavior: 'smooth' })
history.replaceState(null, '', '#summary')  ->  el.focus({ preventScroll: true })
  │
  ▼
During the scroll the observer fires for every section passing the line:
  visible map is updated, but update() returns early because lockedRef is true
  │
  ▼
'scrollend' fires -> release(): lockedRef = false, timer cleared, listener removed
  │
  ▼
'Summary' stays highlighted; the next real scroll hands control back to the observer
```

7. Important technical challenges

Challenge 1: Deciding which section is active, without a scroll handler

Problem. The naive version listens to `scroll`, loops over all headings and calls `getBoundingClientRect()` on each. With 100 headings that is 100 layout reads per scroll event (60+ events per second), and it fights the browser's scrolling thread. Worse, observing only the **headings** fails in the middle of a long section: no heading is on screen, so nothing is "visible", and the highlight jumps or disappears.

Solution.

- Observe the **whole** `<section>`, not the heading. Sections tile the page, so the section that covers the reading line is always known.
- Shrink the observer's root into a **band** with `rootMargin`: the top cuts off the sticky header, the bottom cuts off 60% of the viewport. A section "intersects" when it crosses the top 40% of what the user sees.
- Keep a `Map<id, isIntersecting>`; on each callback pick the **first id in document order** that is visible. If none is (a gap, or a hero above the first section), return `null` and keep the previous id.
- The browser computes intersections off the main thread and calls us only when something crosses the band edge.

```
const sectionObserver = new IntersectionObserver(
  (entries) => {
    for (const e of entries) visible.set(e.target.id, e.isIntersecting);
    update();
  },
  { rootMargin: `-${offset + 8}px 0px -60% 0px` },
);

export function pickActive(ids, visible, atEnd) {
  if (atEnd) return ids.at(-1) ?? null;
  return ids.find((id) => visible.get(id)) ?? null;
}
```

Why "first in document order" and not "biggest intersection ratio": ratios change continuously and two sections can tie, which makes the highlight jitter. "The section under the reading line" is stable and matches how people read top to bottom.

Challenge 2: Click-to-scroll fights the observer

Problem. The user at "Introduction" clicks "Summary". The page smooth-scrolls past six sections; each crosses the reading line, the observer fires, and the highlight flickers Introduction -> Why -> Scroll events -> ... -> Summary. It looks broken. And if the user clicks again mid-scroll, the old "unlock" timer can unlock the new scroll early.

Solution.

- On click, set `activeId` **immediately** (instant feedback) and set `lockedRef = true`.
- The observer keeps updating the `visible` map during the lock (so the data stays correct) but does not call `setActiveId`.
- Unlock on `scrollend`. Fallback timeout of 1s for browsers without `scrollend`, and for the case where no scroll happens at all (target already in place, so `scrollend` never fires).
- Keep the release function in a ref: a second click first releases the first lock, so stale timers/listeners never leak.

```
releaseRef.current?.();          // a second click replaces the first lock
lockedRef.current = true;
setActiveId(id);

const release = () => {
  lockedRef.current = false;
  clearTimeout(timer);
  window.removeEventListener('scrollend', release);
  releaseRef.current = null;
};
const timer = setTimeout(release, UNLOCK_FALLBACK_MS);
window.addEventListener('scrollend', release);
releaseRef.current = release;

el.scrollIntoView({ behavior: smooth && !reduceMotion ? 'smooth' : 'auto', block: 'start' });
history.replaceState(null, '', `#${id}`);
```

Why `replaceState` and not `pushState`: ten TOC clicks would otherwise need ten Back presses to leave the page. Why not just let the browser follow `href="#id"`: it jumps without smooth scroll in many setups, does not lock the highlight, and we would lose control of focus.

Challenge 3: Sticky header offset and the off-by-one at the reading line

Problem. Scenario A: clicking a link puts the heading at `top: 0`, under the 64px sticky header, so the user cannot see the title. Scenario B: we fix that with `scroll-margin-top: 64px`. Now the clicked section starts at 64px and the **previous** section ends at exactly 64px. If the band also started at 64px, the previous section touches the band edge, may count as intersecting, and because it comes first in document order it wins: you click "Edge cases" and "The reading line" lights up.

Solution.

- `scroll-mt-16` (64px) on every section: native CSS, works for TOC clicks, heading anchors and deep links alike.
- Put the band top a few pixels **below** the header: `rootMargin: -(offset + 8)px`. The section that ends at the header is outside the band; the section we scrolled to clearly crosses it.
- One constant (`HEADER_HEIGHT`) feeds the hook; the class `scroll-mt-16` matches it (comment says so).

```
<section id={section.id} tabIndex={-1} className="scroll-mt-16 pt-2 pb-6 outline-none">
```

```
{ rootMargin: `-${offset + 8}px 0px -60% 0px` } // offset = 64
```

Challenge 4: The last section never becomes active, and deep links

Problem. "Summary" is one paragraph long. At the bottom of the page the browser cannot scroll any further, so the Summary top never reaches the reading line; "Clicking a link" stays highlighted forever. Separately, opening `/#summary` does nothing useful: the browser tries its native hash jump **before** React has rendered the sections, finds no element, and stays at the top.

Solution.

- Put an empty **end sentinel** `<div>` after the article and watch it with a second observer (normal viewport). When it is visible **and the page is scrolled**, `pickActive` returns the last id. The `scrollY > 0` check keeps a page that fits on one screen from highlighting the last section on load.
- Deep link: after mount, read `location.hash`, check the id is one of ours, and call `scrollTo(id, { smooth: false })` (instant jump on load, highlight + lock come for free).

```
const endObserver = new IntersectionObserver(([e]) => {
  atEndRef.current = e.isIntersecting && window.scrollY > 0;
  update();
});
if (endRef?.current) endObserver.observe(endRef.current);
```

```
useEffect(() => {
  const id = decodeURIComponent(window.location.hash.slice(1));
  if (id && ids.includes(id)) scrollTo(id, { smooth: false });
}, [ids, scrollTo]);
```

Alternative to the sentinel: add `padding-bottom: 60vh` to the article so every section can reach the line. Simpler, but leaves a big empty area at the end; mention it as a trade-off.

---

8. Implement in the correct order

#	Step	Working result after the step
1	Mock data + `slugify` / `withIds` + asserts	Sections with unique ids, tested in node
2	`Article`: render `<section id>` with h2/h3	Long scrollable page
3	Sticky header + two-column layout, TOC as plain `<a href="#id">` list	Clicking already jumps (browser default)
4	`scroll-mt-16` on sections	Headings no longer hidden under the header

#	Step	Working result after the step
5	`useScrollSpy`: one IntersectionObserver with the band, `pickActive`, `activeId`	Highlight follows scrolling
6	Highlight styles, `aria-current`, h3 indentation, parent emphasis	Looks and reads like a real TOC
7	`scrollTo`: preventDefault, smooth scroll, `replaceState` hash, focus	Smooth navigation with shareable URL
8	Click lock with `scrollend` + timeout fallback, release on second click	No flicker during click scrolls
9	End sentinel for the short last section	Bottom of page highlights the last section
10	Deep link on mount (`smooth: false`)	`/#id` opens at the right place
11	Reduced motion, cmd/ctrl-click passthrough, TOC self-scroll	Polish and accessibility
12	Walk the interaction table from step 2	Every row checked

Steps 1-5 already give a working scrollspy (about 30 minutes). Everything after is a layer that can be cut if time runs out.

---

9. Cover all states and edge cases

State / edge case	What happens
Initial (top of page)	First section active (initial state + observer's first callback)
Page restored mid-scroll on reload	IntersectionObserver always fires once on `observe()`, so the right section is picked immediately
Middle of a long section	That section still crosses the band, stays active
Gap above the first section (hero, title)	Nothing visible -> `pickActive` returns `null` -> previous id kept
Short last section at the bottom	End sentinel visible -> last id
Whole article fits on screen	`scrollY === 0` -> end rule ignored, first section active
Click (smooth scroll in progress)	Locked: clicked id stays highlighted
Click on the section already at the top	No scroll happens, `scrollend` never fires -> 1s timeout releases the lock
Two fast clicks	First lock released before the second starts; ends on the second id
Browser without `scrollend` (older Safari)	Timeout fallback releases the lock
Deep link to a valid id	Instant scroll on mount, highlighted
Deep link to unknown id / encoded chars	Ignored if not in `ids`; `decodeURIComponent` handles `%20` etc.
Duplicate titles	`withIds` adds `-1`, `-2` and skips suffixes that already exist

State / edge case	What happens
Title with only symbols (`???`)	Slug falls back to `section` (then `section-1`, ...)
Empty article (`ids = []`)	`activeId` is `null`, no observers on missing elements, TOC renders an empty list
Element missing for an id	`getElementById` returns null: not observed, `scrollTo` returns early
Reduced motion	`behavior: 'auto'` (instant jump)
Cmd/Ctrl/Shift-click	Not prevented: opens in new tab/window as usual
Content changes (new `ids` array)	Effect cleanup disconnects observers and re-observes the new sections
Unmount during a click scroll	Pending timer and `scrollend` listener are released

10. Final structure

```
_86_scrollspy_table_of_contents/
├── index.html
├── package.json
├── vite.config.js
└── src/
    ├── main.jsx
    ├── index.css
    ├── App.jsx                      layout, ids, deep link
    ├── data/
    │   └── article.js               mock content
    ├── utils/
    │   └── toc.js                   slugify, withIds, pickActive (pure)
    ├── hooks/
    │   └── useScrollSpy.js          observers, activeId, scrollTo + lock
    ├── components/
    │   ├── Article.jsx              sections + end sentinel
    │   └── TableOfContents.jsx      links, highlight, self-scroll
    └── toc.test.mjs                 node asserts for the pure utils
```

11. Full working code

Setup: `npm create vite@latest` (React, JavaScript), then `npm i tailwindcss @tailwindcss/vite`.

`vite.config.js`

```
import { defineConfig } from 'vite';
import react from '@vitejs/plugin-react-swc';
```

```
import tailwindcss from '@tailwindcss/vite';

export default defineConfig({
  plugins: [react(), tailwindcss()],
});
```

src/main.jsx

```
import { StrictMode } from 'react';
import { createRoot } from 'react-dom/client';
import App from './App';
import './index.css';

createRoot(document.getElementById('root')).render(
  <StrictMode>
    <App />
  </StrictMode>,
);
```

src/index.css

```
@import "tailwindcss";

body {
  @apply bg-slate-50 text-slate-900;
}
```

src/data/article.js

```
// Mock content. In a real app this comes from a CMS / markdown with the same shape.
const filler = (topic, count) =>
  Array.from(
    { length: count },
    (_, i) =>
      `${topic} (${i + 1}/${count}). The browser decides what is visible, the hook only listens. ` +
      'Long paragraphs make the page scroll so we can watch the table of contents follow the reader. '
+
      'Each section is a block of content that starts at its heading and ends where the next heading
begins.',
  );

export const ARTICLE = {
  title: 'Building a Scrollspy',
  sections: [
```

```
    { title: 'Introduction', level: 2, paragraphs: filler('Introduction', 4) },
    { title: 'Why IntersectionObserver?', level: 2, paragraphs: filler('Why IntersectionObserver', 3)
},
    { title: 'Scroll events', level: 3, paragraphs: filler('Scroll events', 3) },
    { title: 'Observer basics', level: 3, paragraphs: filler('Observer basics', 4) },
    { title: 'Picking the active section', level: 2, paragraphs: filler('Picking the active section',
5) },
    { title: 'The reading line', level: 3, paragraphs: filler('The reading line', 3) },
    { title: 'Edge cases', level: 3, paragraphs: filler('Edge cases', 2) },
    { title: 'Clicking a link', level: 2, paragraphs: filler('Clicking a link', 4) },
    { title: 'Edge cases', level: 3, paragraphs: filler('Edge cases again', 2) }, // duplicate title ->
unique id
    { title: 'Summary', level: 2, paragraphs: filler('Summary', 1) }, // short last section
  ],
};
```

src/utils/toc.js

```
// "Getting Started!" -> "getting-started"
export function slugify(text) {
  const slug = text
    .toLowerCase()
    .normalize('NFKD')
    .replace(/[̀-ͯ]/g, '') // strip accents: "Café" -> "cafe"
    .replace(/[^a-z0-9\s-]/g, '')
    .trim()
    .replace(/[\s-]+/g, '-');
  return slug || 'section';
}

// Adds a unique id to every section and links each h3 to the h2 above it.
export function withIds(sections) {
  const used = new Set();
  let parentId = null;
  return sections.map((section) => {
    const base = slugify(section.title);
    let id = base;
    for (let n = 1; used.has(id); n++) id = `${base}-${n}`;
    used.add(id);
    if (section.level === 2) parentId = id;
    return { ...section, id, parentId: section.level === 3 ? parentId : null };
  });
}

// ids are in document order. visible: Map<id, boolean> of sections crossing the reading line.
// Returns null when nothing is visible, so the caller keeps the previous active id.
```

```
export function pickActive(ids, visible, atEnd) {
  if (atEnd) return ids.at(-1) ?? null;
  return ids.find((id) => visible.get(id)) ?? null;
}
```

src/hooks/useScrollSpy.js

```
import { useCallback, useEffect, useRef, useState } from 'react';
import { pickActive } from '../utils/toc';

const UNLOCK_FALLBACK_MS = 1000; // browsers without `scrollend`, or when no scroll happens

// ids: section ids in document order. offset: height of the sticky header in px.
export function useScrollSpy(ids, { offset = 0, endRef } = {}) {
  const [activeId, setActiveId] = useState(ids[0] ?? null);
  const visibleRef = useRef(new Map());
  const atEndRef = useRef(false);
  const lockedRef = useRef(false);
  const releaseRef = useRef(null);

  useEffect(() => {
    const visible = visibleRef.current;
    visible.clear();
    const update = () => {
      if (lockedRef.current) return; // a click-scroll is running: keep the clicked id
      const next = pickActive(ids, visible, atEndRef.current);
      if (next) setActiveId(next);
    };

    // The "reading line" is a band just below the sticky header (top 40% of the viewport).
    // +8px so the section that ends exactly at the header (after scroll-margin) does not win.
    const sectionObserver = new IntersectionObserver(
      (entries) => {
        for (const e of entries) visible.set(e.target.id, e.isIntersecting);
        update();
      },
      { rootMargin: `-${offset + 8}px 0px -60% 0px` },
    );
    ids.forEach((id) => {
      const el = document.getElementById(id);
      if (el) sectionObserver.observe(el);
    });

    // Short last sections can never reach the reading line, so "end of article visible" picks the last
one.
    const endObserver = new IntersectionObserver(([e]) => {
```

```
      atEndRef.current = e.isIntersecting && window.scrollY > 0; // a page that fits on screen keeps
the first section
      update();
    });
    if (endRef?.current) endObserver.observe(endRef.current);

    return () => {
      sectionObserver.disconnect();
      endObserver.disconnect();
    };
  }, [ids, offset, endRef]);

  // Cancel a pending lock on unmount.
  useEffect(() => () => releaseRef.current?.(), []);

  const scrollTo = useCallback((id, { smooth = true } = {}) => {
    const el = document.getElementById(id);
    if (!el) return;

    releaseRef.current?.(); // a second click replaces the first lock
    lockedRef.current = true;
    setActiveId(id);

    const release = () => {
      lockedRef.current = false;
      clearTimeout(timer);
      window.removeEventListener('scrollend', release);
      releaseRef.current = null;
    };
    const timer = setTimeout(release, UNLOCK_FALLBACK_MS);
    window.addEventListener('scrollend', release);
    releaseRef.current = release;

    const reduceMotion = window.matchMedia('(prefers-reduced-motion: reduce)').matches;
    el.scrollIntoView({ behavior: smooth && !reduceMotion ? 'smooth' : 'auto', block: 'start' });
    history.replaceState(null, '', `#${id}`);
    el.focus({ preventScroll: true }); // keyboard and screen reader users continue reading from here
  }, []);

  return { activeId, scrollTo };
}
```

src/components/Article.jsx

```
export default function Article({ title, sections, endRef }) {
  return (
```

```
    <article className="max-w-none">
      <h1 className="mb-6 text-3xl font-bold">{title}</h1>
      {sections.map((section) => {
        const Heading = section.level === 2 ? 'h2' : 'h3';
        return (
          // scroll-mt-16 = sticky header height, so the heading is not hidden under it
          <section key={section.id} id={section.id} tabIndex={-1} className="scroll-mt-16 pt-2 pb-6
outline-none">
            <Heading className={section.level === 2 ? 'mb-3 text-2xl font-semibold' : 'mb-2 text-lg
font-semibold'}>
              <a href={`#${section.id}`} className="hover:underline">
                {section.title}
              </a>
            </Heading>
            {section.paragraphs.map((text, i) => (
              <p key={i} className="mb-3 leading-7 text-slate-700">
                {text}
              </p>
            ))}
          </section>
        );
      })}
      <div ref={endRef} aria-hidden="true" />
    </article>
  );
}
```

src/components/TableOfContents.jsx

```
import { useEffect, useRef } from 'react';

export default function TableOfContents({ items, activeId, onSelect }) {
  const navRef = useRef(null);
  const active = items.find((item) => item.id === activeId);
  const activeParentId = active?.parentId;

  // Long TOC: keep the active link inside the TOC's own scroll box.
  // Sets scrollTop by hand: scrollIntoView here would cancel the page's smooth scroll.
  useEffect(() => {
    const nav = navRef.current;
    const link = nav?.querySelector('[aria-current="location"]');
    if (!link) return;
    const outside =
      link.offsetTop < nav.scrollTop ||
      link.offsetTop + link.offsetHeight > nav.scrollTop + nav.clientHeight;
    if (outside) nav.scrollTop = link.offsetTop - nav.clientHeight / 2;
```

```
  }, [activeId]);

  return (
    <nav
      ref={navRef}
      aria-label="Table of contents"
      className="sticky top-20 max-h-[calc(100vh-6rem)] overflow-y-auto"
    >
      <p className="mb-2 text-xs font-semibold uppercase tracking-wide text-slate-500">On this page</p>
      <ul className="space-y-1 border-l border-slate-200">
        {items.map((item) => {
          const isActive = item.id === activeId;
          const isParentOfActive = item.id === activeParentId;
          return (
            <li key={item.id}>
              <a
                href={`#${item.id}`}
                aria-current={isActive ? 'location' : undefined}
                onClick={(e) => {
                  if (e.metaKey || e.ctrlKey || e.shiftKey) return; // let "open in new tab" work
                  e.preventDefault();
                  onSelect(item.id);
                }}
                className={[
                  '-ml-px block border-l-2 py-1 text-sm transition-colors',
                  item.level === 3 ? 'pl-7' : 'pl-3',
                  isActive
                    ? 'border-blue-600 font-medium text-blue-700'
                    : isParentOfActive
                      ? 'border-transparent text-slate-900'
                      : 'border-transparent text-slate-500 hover:text-slate-900',
                ].join(' ')}
              >
                {item.title}
              </a>
            </li>
          );
        })}
      </ul>
    </nav>
  );
}
```

`src/App.jsx`

```
import { useEffect, useMemo, useRef } from 'react';
import { ARTICLE } from './data/article';
import { withIds } from './utils/toc';
import { useScrollSpy } from './hooks/useScrollSpy';
import Article from './components/Article';
import TableOfContents from './components/TableOfContents';

const HEADER_HEIGHT = 64; // h-16, keep in sync with scroll-mt-16 on sections

export default function App() {
  const sections = useMemo(() => withIds(ARTICLE.sections), []);
  const ids = useMemo(() => sections.map((s) => s.id), [sections]);
  const endRef = useRef(null);
  const { activeId, scrollTo } = useScrollSpy(ids, { offset: HEADER_HEIGHT, endRef });

  // Deep link: the page renders after the browser's own #hash jump, so do it ourselves once.
  useEffect(() => {
    const id = decodeURIComponent(window.location.hash.slice(1));
    if (id && ids.includes(id)) scrollTo(id, { smooth: false });
  }, [ids, scrollTo]);

  return (
    <>
      <header className="sticky top-0 z-10 flex h-16 items-center border-b border-slate-200 bg-white/90
px-6 backdrop-blur">
        <span className="font-semibold">Docs</span>
      </header>
      <div className="mx-auto grid max-w-6xl gap-10 px-6 py-8 lg:grid-cols-[1fr_240px]">
        <main>
          <Article title={ARTICLE.title} sections={sections} endRef={endRef} />
        </main>
        <aside className="hidden lg:block">
          <TableOfContents items={sections} activeId={activeId} onSelect={scrollTo} />
        </aside>
      </div>
    </>
  );
}
```

`src/toc.test.mjs` (run with `node src/toc.test.mjs`)

```
import assert from 'node:assert/strict';
import { slugify, withIds, pickActive } from './utils/toc.js';
```

```
// slugify
assert.equal(slugify('Why IntersectionObserver?'), 'why-intersectionobserver');
assert.equal(slugify('  Café & Crème  '), 'cafe-creme');
assert.equal(slugify('a -- b'), 'a-b');
assert.equal(slugify('???'), 'section');

// withIds: unique ids and parent links
const items = withIds([
  { title: 'Intro', level: 2 },
  { title: 'Edge cases', level: 3 },
  { title: 'Edge cases', level: 3 },
  { title: 'Edge cases 1', level: 2 }, // slug "edge-cases-1" already taken by the duplicate above
  { title: 'Child', level: 3 },
]);
assert.deepEqual(items.map((i) => i.id), ['intro', 'edge-cases', 'edge-cases-1', 'edge-cases-1-1',
'child']);
assert.deepEqual(items.map((i) => i.parentId), [null, 'intro', 'intro', null, 'edge-cases-1-1']);

// pickActive
const ids = ['a', 'b', 'c'];
assert.equal(pickActive(ids, new Map([['b', true], ['c', true]]), false), 'b'); // first in document
order
assert.equal(pickActive(ids, new Map([['a', false]]), false), null); // nothing visible -> keep
previous
assert.equal(pickActive(ids, new Map([['a', true]]), true), 'c'); // end of article visible
assert.equal(pickActive([], new Map(), true), null);

console.log('toc logic: all asserts passed');
```

---

12. Wrap-up

Edge cases handled

- Long sections (observe sections, not headings), gaps (keep previous id).
- Short last section (end sentinel), page that fits on one screen.
- Sticky header (scroll-margin + band offset, with the 8px off-by-one guard).
- Click flicker (lock until `scrollend`, timeout fallback, second click replaces the lock).
- Deep links after client render, unknown hashes, duplicate and symbol-only titles.
- Reduced motion, modifier-clicks, focus after navigation, `aria-current`.

Trade-offs to mention

Decision	Why	Cost
IntersectionObserver band instead of `scroll` + `getBoundingClientRect`	No per-pixel work, no forced layout, scales to many sections	Slightly harder to reason about (rootMargin), callbacks only fire on crossings
"First visible in document order"	Stable, matches reading direction	A tiny section between two big ones may be active only briefly
End sentinel	Last section reachable without changing layout	When the last 2-3 sections are all short, the ones between are skipped while scrolling (still clickable)
Lock until `scrollend`	No flicker on click	If the user scrolls away mid-animation, the highlight catches up on their next scroll movement, not instantly
`replaceState`	Shareable URL without polluting history	Back button does not return to the previous section
Flat sections with `parentId`	Simple order-based picking	A collapsible nested TOC needs a tree built from it
Data-driven ids	Stable, readable URLs	Renaming a heading breaks old links (could keep an alias map)

What to add with more time

- **Mobile TOC:** a collapsible "On this page" button under the header that shows the active title.
- **Reading progress:** a thin bar showing scroll progress, or per-section progress in the TOC.
- **Tests:** React Testing Library with a mocked `IntersectionObserver` (trigger entries by hand and assert `aria-current`); Playwright for real scrolling, deep links and the click lock.
- **Content from markdown / CMS:** generate ids at build time (`rehype-slug`) and pass them in; or scan `article.querySelectorAll('h2, h3')` when the HTML is given.
- **Collapsible groups** in the TOC (expand only the active h2's children).
- **Dynamic content:** a `MutationObserver` or a `ids` change when sections load lazily (images changing heights are already handled by the observer).
- **Analytics:** report which sections were actually read (the same observer data).

---

13. Counter questions the interviewer may ask

Requirements

Q1. Why highlight the section at the top of the viewport and not the one that takes most of the screen?

People read from the top down, and docs sites (MDN, Stripe, React docs) all use the top reading line, so it matches expectations. "Most visible" changes continuously and two sections can tie, which makes the

highlight jump back and forth. If the interviewer prefers "most visible", I would use `threshold: [0, 0.25, 0.5, 0.75, 1]` and pick the max `intersectionRatio`.

Q2. What if the content is given as HTML (markdown already rendered) and not as an array?

Then I would build the TOC from the DOM: after mount, `articleRef.current.querySelectorAll('h2, h3')`, read `textContent`, `tagName` and existing `id` (or set one with `withIds`). The hook does not change, it only needs `ids`. I would observe each heading's surrounding block or wrap content between headings into sections at render time.

State design

Q3. Why keep the visibility map in a ref and not in state?

It changes on every observer callback, but the UI only needs the picked id. Storing it in state would re-render the TOC on every crossing even when the active id stays the same. The ref holds the raw data; `activeId` is the only rendered state.

Q4. Why is `lockedRef` a ref and not state?

It is read inside the observer callback, which is created once per effect. A state value there would be stale (closure from the render that created the observer). A ref is always current and changing it should not re-render anything.

Q5. Why does `pickActive` return `null` instead of the first id when nothing is visible?

`null` means "no new information, keep what you have". In a gap between sections, or above the first section, jumping back to the first id would be wrong. Returning `null` keeps the decision with the caller, and it stays a pure, testable function.

Challenge: picking the active section

Q6. Why not a `scroll` listener with throttle?

It works, but every call does a layout read per heading, it runs on the main thread during the scroll, and the throttle adds lag (or misses the last position unless you add a trailing call). IntersectionObserver is computed by the browser and only calls us on crossings, which is both cheaper and more accurate. A throttled scroll handler is the fallback I would mention for environments without IO.

Q7. Why observe sections and not the headings?

A heading is 30px tall. While reading a long section no heading is in the band, so a heading-based observer has nothing visible and the highlight goes stale or blank. Sections cover the whole page, so there is always one under the reading line.

Q8. Explain the `rootMargin` value.

`-(64+8)px 0px -60% 0px` shrinks the root (the viewport): the top 72px (header plus guard) and the bottom 60% are ignored. What remains is a band from 72px to 40% of the height. A section is "intersecting" when

any part of it is inside that band. Negative margins only shrink; percentages are of the root size, so it adapts to the window height.

Q9. What if the user resizes the window or images load and change heights?

Nothing to do: IntersectionObserver re-computes on layout changes, and the percentage margin follows the new height. That is another win over caching heading offsets, which would go stale.

Challenge: click lock

Q10. What if the user starts scrolling with the wheel during the smooth scroll?

The browser cancels the smooth scroll and the user's scroll continues; `scrollend` fires when they stop, the lock is released, and the next crossing updates the highlight. To be even more responsive I could release the lock on `wheel`, `touchstart` or `keydown` as well.

Q11. Why the 1 second timeout if `scrollend` exists?

Two reasons: older Safari has no `scrollend`, and if the target is already in place no scroll happens so `scrollend` never fires. Without the timeout the observer would stay locked forever. The timeout is cleared when `scrollend` comes first.

Q12. Why call `el.focus({ preventScroll: true })` after the click?

Without it, keyboard focus stays on the TOC link, so the next Tab goes to the next TOC link instead of the content the user jumped to, and screen readers do not move. `tabIndex={-1}` makes the section focusable by script only; `preventScroll` stops focus from cancelling our smooth scroll.

Challenge: header offset and bottom of page

Q13. Why `scroll-margin-top` in CSS instead of computing `window.scrollTo(top - 64)`?

`scroll-margin-top` is respected by every way of reaching the section: `scrollIntoView`, native `#hash` jumps, heading anchor links and "find in page". A manual offset would only fix our own clicks. It also keeps `scrollIntoView` (and its smooth behavior) instead of re-implementing it.

Q14. What if the header height changes (responsive, or a banner appears)?

Measure it instead of a constant: a `ResizeObserver` on the header sets a CSS variable `--header-h` used by `scroll-margin-top: var(--header-h)` and passed as `offset` to the hook. The hook already re-creates the observer when `offset` changes.

Q15. What if the last three sections are all short?

With the sentinel, the moment the end of the article is visible the last section wins, so the middle two are skipped while scrolling (they still highlight when clicked, thanks to the lock). The other fix is bottom padding (`pb-[60vh]`) so every section can reach the reading line, at the cost of empty space at the end. I would ask the designer which they prefer.

Implementation steps

Q16. Why use real `<a href="#id">` links and then `preventDefault`?

Real anchors work without JS, can be copied, middle-clicked and opened in a new tab, and are announced as links. We only take over the plain left click to add smooth scrolling, the lock and focus. Modifier clicks fall through to the browser.

Q17. Why does the deep link need code? Browsers jump to `#id` on their own.

The native jump happens when the document loads. With a client-rendered React app the sections do not exist yet at that moment, so the browser finds nothing. After mount we do the jump ourselves with `smooth: false`, which also sets the highlight and the lock.

Q18. Why is the observer created in `useEffect` and not in a ref callback per section?

One observer for all sections is cheaper than one per element, and the effect has a single cleanup (`disconnect()`) that runs when `ids` or `offset` change or on unmount. Child DOM is committed before the parent's effect runs, so `getElementById` finds every section.

Performance

Q19. What if the article has 1,000 sections?

One IntersectionObserver handles thousands of targets efficiently. `pickActive` is O(n) per callback, which is still trivial; if it showed up in a profile I would keep a sorted set of visible indexes and take the smallest. The bigger cost would be rendering 1,000 sections: I would virtualize or lazy-render far sections, and make the TOC collapsible so only the active group is expanded.

Q20. How many re-renders happen while scrolling the whole page?

About one per section boundary: `setActiveId` is only called with a new id when a section crosses the band, and React bails out when the value is the same. The article itself never re-renders because its props (`sections`, `endRef`) are stable.

Trade-offs

Q21. Why not update the URL hash while scrolling, like some docs sites do?

Writing to history on every section change is noisy, fills the history if done with `pushState`, and in some browsers `replaceState` called many times per second is throttled with warnings. Updating on click is enough for sharing. If required, I would call `replaceState` in the same place `setActiveId` changes, never on raw scroll.

Q22. Would you put this in a library like `react-scrollspy` or use a ready-made one?

In a product I would check the existing design system first. In an interview I write it, because the hook is about 70 lines and the interesting part (band, ordering, lock, end sentinel) is exactly what is being tested. The hook API (`ids`, `offset` -> `activeId`, `scrollTo`) is the same shape a library would expose, so swapping later is easy.

Q23. How would you test the hook?

Pure parts (`slugify` , `withIds` , `pickActive`) with plain unit tests, as in `toc.test.mjs` . For the hook, mock `IntersectionObserver` in jsdom, capture the callback, call it with fake entries and assert `aria-current` on the TOC link; also assert that during a `scrollTo` lock the callback does not change the highlight. Real scrolling, `scrollend` and deep links go to a Playwright test.